TYPOSTÉROÏDES

TD2 – Taper pour survivre

Un jeu de frappe en Python avec pygame

Séance de 2 h

Version corrigée

Objectifs de la séance

Au TD1, les mots fonçaient sur le vaisseau sans qu'on puisse rien faire. Aujourd'hui, le jeu devient **jouable** : on tape un mot pour le détruire, le canon tire, on marque des points, et la difficulté monte. Le fichier obtenu sera **la base de tous les TP**.

- structurer un jeu avec une **machine à états** (menu, partie, fin) ;
- lire le **clavier** caractère par caractère ;
- manipuler des **chaînes** : indices, tranches (*slicing*) ;
- écrire un **algorithme de recherche du minimum** ;
- créer des objets **à durée de vie limitée** (lasers) ;
- calculer score, combo, précision, mots par minute ;
- régler une **difficulté progressive** (suites arithmétiques et géométriques).

Table des matières

1 Point de départ **1**

2 Comment fonctionne pygame (suite) **2**

2.1 Le clavier en détail . . . 2
2.2 Transparence et calques . . . 2
2.3 Placer un texte : `get_rect` et les arguments nommés . . . 3
2.4 Faire tourner un vecteur . . . 3
2.5 Organiser un jeu : la machine à états . . . 3

3 Construire le jeu, étape par étape **3**

4 Pour aller plus loin (si vous avez fini) **11**

5 Backlog : tout ce qu'on peut ajouter au jeu **12**

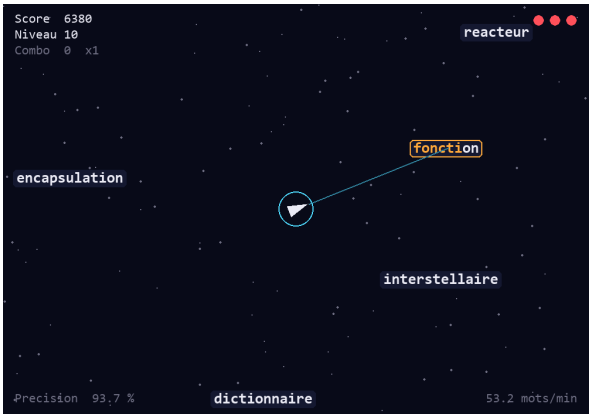
6 Feuille de route des TP (10 h) **14**

1 Point de départ

Le fichier `td2_eleve.py` reprend la correction du TD1 et ajoute de nouveaux `TODO n`. Comme au TD1, **le numéro du TODO est celui de l'étape**.

Les règles du jeu final :

- taper la **première lettre** d'un mot le **verrouille** (cadre orange) ;
- chaque bonne lettre **tire un laser** et fait **reculer** le mot ;
- le mot explose quand il est entièrement tapé ;
- une faute **casse le combo** ; `Retour arrière` libère la cible ;
- tous les 10 mots, on passe au **niveau** suivant : les mots sont plus longs, plus rapides et plus nombreux.



Le jeu à la fin du TD2 (niveau 10).

Ce qui est nouveau dans le fichier

Élément	Rôle
`MOTS` (dictionnaire)	les mots rangés par longueur : `"courts"`, `"moyens"`, `"longs"`, `"tres_longs"`
`Mot.tape`	nombre de lettres déjà tapées (0 au départ)
`Joueur.direction`	vecteur unitaire : où pointe le canon
classe `Laser`	un trait lumineux qui s'efface en 0,12 s
`Jeu.etat`	`"menu"`, `"jeu"` ou `"game_over"`
`Jeu.cible`	le `Mot` verrouillé, ou `None`
`score`, `combo`, `niveau`...	les statistiques de la partie
`Jeu.ecrire(...)`	petit utilitaire pour afficher un texte en une ligne

2 Comment fonctionne pygame (suite)

2.1 Le clavier en détail

Il y a **deux façons** de lire le clavier avec pygame, pour deux usages différents :

	Événements `KEYDOWN`	**État `pygame.key.get_pressed()`**
Question posée	« Quelle touche vient d'être *enfoncée* ? »	« Quelles touches sont *maintenues* en ce moment ? »
Combien de fois	une fois par appui	à chaque image tant que la touche est enfoncée
Usage typique	taper du texte, valider, mettre en pause	déplacer un vaisseau en continu
Dans notre jeu	**oui** (un mot = une suite d'appuis)	non

Un événement `KEYDOWN` porte deux informations bien distinctes :

- `evenement.key` : la **touche**, comparée à des constantes : `pygame.K_RETURN`, `K_KP_ENTER` (Entrée du pavé numérique), `K_ESCAPE`, `K_BACKSPACE`, `K_SPACE`, `K_a`...
- `evenement.unicode` : le **caractère** produit, en tenant compte de la disposition du clavier (AZERTY / QWERTY) et de `Maj` : `"a"`, `"A"`, `"?"`... Pour Entrée c'est `"\r"`, pour Échap

`"\x1b"`, pour une flèche c'est la chaîne vide `""`.

Pourquoi `unicode` et pas `key` pour taper ?

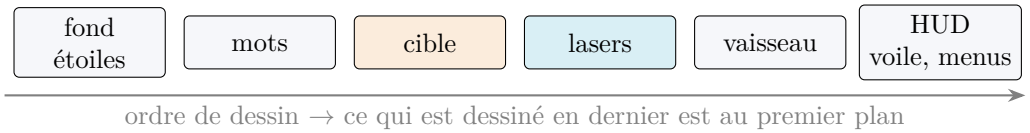
Sur un clavier AZERTY, la touche marquée « A » est à l'emplacement du « Q » d'un QWERTY. Selon le système, `key` peut renvoyer `K_q` ! `unicode` renvoie toujours le caractère réellement tapé : c'est ce qu'il faut pour un jeu de frappe. On filtre avec `str.isalpha()` (vrai seulement pour des lettres) et on passe en minuscules avec `str.lower()`.
Les lettres accentuées tapées avec une *touche morte* (^ puis e) passent par un autre événement, `pygame.TEXTINPUT` : ce sera une piste du TP1.

2.2 Transparence et calques

Une `Surface` normale est opaque. Pour un voile semi-transparent (l'écran de game over), on crée une Surface avec un **canal alpha** (opacité de 0 à 255) :

```
voile = pygame.Surface((LARGEUR, HAUTEUR), pygame.SRCALPHA)
voile.fill((0, 0, 0, 170))          # noir, opaque à 170/255
self.ecran.blit(voile, (0, 0))      # assombrit tout ce qui est déjà dessiné
```

L'image finale est construite par **couches**, du fond vers l'avant :



2.3 Placer un texte : `get_rect` et les arguments nommés

`image.get_rect(center=(x, y))` renvoie le rectangle de l'image placé de sorte que son *centre* soit en (x, y). On peut utiliser n'importe quel point remarquable : `topleft`, `midleft`, `bottomright`… Le fichier fournit un utilitaire qui transmet cet argument tel quel grâce à la syntaxe `**position` :

```
def ecrire(self, texte, police, couleur, **position):
    image = police.render(texte, True, couleur)
    self.ecran.blit(image, image.get_rect(**position))

self.ecrire("Score 120", self.police_hud, BLANC, topleft=(20, 15))
self.ecrire("GAME OVER", self.police_titre, ROUGE, center=(500, 230))
```

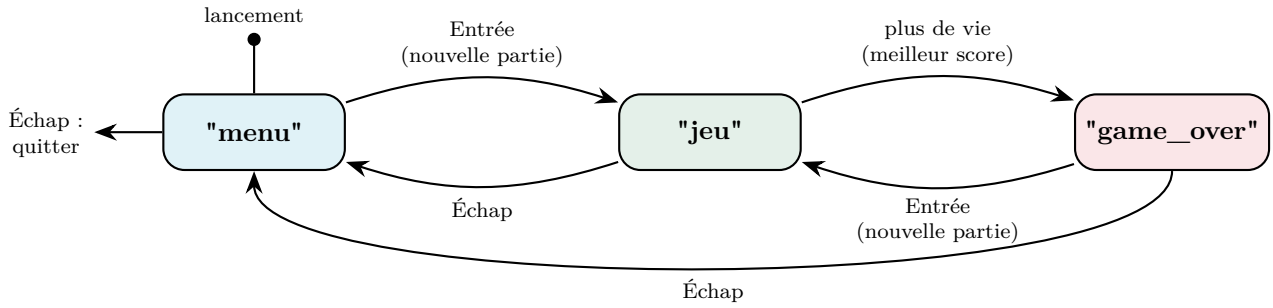
`**position` récupère tous les arguments nommés dans un dictionnaire (`{"topleft": (20, 15)}`), et `get_rect(**position)` les redistribue.

2.4 Faire tourner un vecteur

`v.rotate(angle)` renvoie le vecteur v tourné de `angle` **degrés**. Comme l'axe y descend, un angle positif tourne dans le sens *horaire* à l'écran. `v.angle_to(w)` donne l'angle de v vers w. C'est tout ce qu'il faut pour orienter un canon : pas besoin de `atan2`.

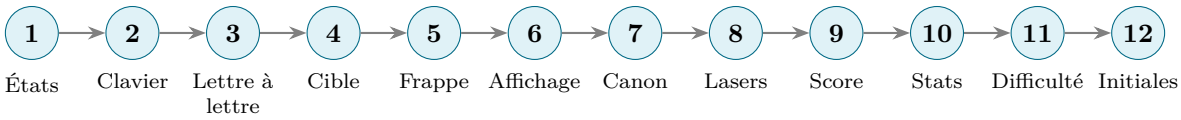
2.5 Organiser un jeu : la machine à états

Un vrai jeu n'est pas qu'une partie : il y a un menu, une pause, un écran de fin… On le modélise par une **machine à états** : une variable `self.etat` et des **transitions** déclenchées par des événements. Chacune des trois méthodes de la boucle commence alors par « dans quel état suis-je ? ».



3 Construire le jeu, étape par étape

Même méthode qu'au TD1 : **une étape, on lance, on vérifie**. Si un test échoue, on corrige avant de passer à la suite.



Étape 1 – La machine à états `TODO 1a` `TODO 1b` `TODO 1c` 15 min

Lisez `dessiner` et le début de `mettre_a_jour` : ils testent déjà `self.etat`. Il reste à écrire les **transitions** (voir le diagramme de la partie 2).

1. `TODO 1a` : dans `gerer_touche`, état `"menu"` : Entrée lance une nouvelle partie et passe dans l'état `"jeu"` ; Échap quitte le programme. La variable `entree` (déjà calculée) vaut `True` pour les deux touches Entrée.
2. `TODO 1b` : état `"game_over"` : Entrée relance, Échap revient au menu.
3. `TODO 1c` : à la fin de `mettre_a_jour`, quand le joueur n'a plus de vie, mémorisez le meilleur score (`max`) et passez dans l'état `"game_over"`.

✓ Testez : lancez le jeu, vous devez voir...

Le menu « TYPOSTEROIDES » au lancement. Entrée → les mots arrivent. Après 3 chocs, l'écran de fin s'affiche par-dessus la partie figée. Entrée relance, Échap ramène au menu, Échap dans le menu ferme le jeu.

Correction

```
if self.etat == "menu":
    if entree:
        self.nouvelle_partie()
        self.etat = "jeu"
    elif evenement.key == pygame.K_ESCAPE:
        self.en_cours = False
# ...
elif self.etat == "game_over":
    if entree:
        self.nouvelle_partie()
        self.etat = "jeu"
    elif evenement.key == pygame.K_ESCAPE:
        self.etat = "menu"

# fin de mettre_a_jour :
```

```python
if not self.joueur.est_vivant():
    self.meilleur_score = max(self.meilleur_score, self.score)
    self.etat = "game_over"
```

Étape 2 – Lire le clavier TODO 2 — 10 min

Dans l'état `"jeu"` : Échap revient au menu, `Retour arrière` appelle `self.changer_de_cible()`, et **une lettre** appelle `self.gerer_frappe(lettre)` avec la lettre **en minuscule**.

Pour vérifier, ajoutez provisoirement `print("frappe :", lettre)` au début de `gerer_frappe`.

✓ Testez : lancez le jeu, vous devez voir...

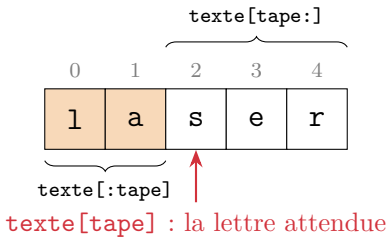
Dans la console, chaque lettre tapée s'affiche en minuscule (essayez avec `Maj`) ; les chiffres, flèches et la barre d'espace n'affichent rien.

Correction

```python
elif self.etat == "jeu":
    if evenement.key == pygame.K_ESCAPE:
        self.etat = "menu"
    elif evenement.key == pygame.K_BACKSPACE:
        self.changer_de_cible()
    elif evenement.unicode.isalpha():
        self.gerer_frappe(evenement.unicode.lower())
```

Étape 3 – Un mot qui se tape lettre à lettre TODO 3a TODO 3b TODO 3c — 15 min

Chaque `Mot` retient combien de lettres ont déjà été tapées dans `self.tape`. Avec le mot `"laser"` et `tape = 2` :



1. TODO 3a : `lettre_attendue()` renvoie `self.texte[self.tape]`.
2. TODO 3b : `est_termine()` est vrai quand `tape` vaut la longueur du texte.
3. TODO 3c : `taper(lettre)` : si c'est la lettre attendue, on avance `tape`, le mot **recule** de `RECUL` pixels (à l'opposé de `self.direction`) et on renvoie `True` ; sinon on renvoie `False`.

✓ Testez : lancez le jeu, vous devez voir...

On peut tester une classe **sans lancer le jeu** ! Dans un terminal Python ouvert dans le dossier du fichier :

```
>>> import pygame, td2_eleve as t     # le jeu ne démarre pas : merci __main__ !
>>> m = t.Mot("laser", pygame.Vector2(500, 350), 50)
>>> m.taper("l"), m.taper("x"), m.taper("a")
(True, False, True)
>>> m.tape, m.lettre_attendue(), m.est_termine()
(2, 's', False)
```

Correction

```python
def lettre_attendue(self):
    return self.texte[self.tape]

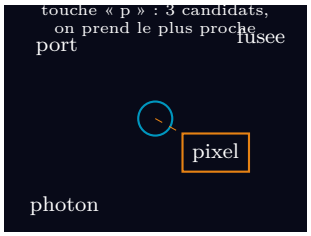
def est_termine(self):
    return self.tape == len(self.texte)

def taper(self, lettre):
    if lettre == self.lettre_attendue():
        self.tape += 1
        self.position -= self.direction * RECUL   # recul : sens opposé
        return True
    return False
```

Étape 4 – Choisir la cible `TODO 4` 10 min

Quand aucun mot n'est verrouillé et que le joueur tape « p », quel mot viser ? Le plus **dangereux**, c'est-à-dire **le plus proche du vaisseau** parmi ceux qui commencent par « p ». C'est l'algorithme classique du **minimum** :

1. `meilleur = None`, `meilleure_distance = math.inf` ;
2. pour chaque mot qui commence par la lettre, si sa distance est plus petite, il devient le meilleur ;
3. renvoyer `meilleur` (qui vaut `None` si aucun mot ne convient).



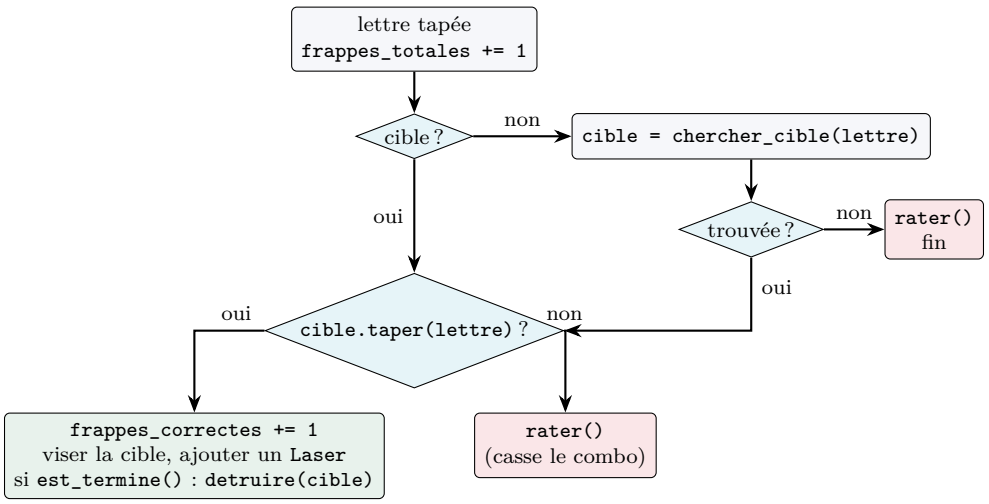
Écrivez `chercher_cible(lettre)`. *Pour les curieux*, la version compacte en une ligne utilise `min` avec une `key` : `min(candidats, key=lambda m: m.position.distance_to(centre), default=None)`.

Correction

```python
def chercher_cible(self, lettre):
    meilleur = None
    meilleure_distance = math.inf
    for mot in self.mots:
        if mot.texte[0] == lettre:
            distance = mot.position.distance_to(self.joueur.position)
            if distance < meilleure_distance:
                meilleur = mot
                meilleure_distance = distance
    return meilleur
```

Étape 5 – Gérer une frappe `TODO 5` 15 min

C'est le cœur du gameplay. Suivez l'organigramme :



Le laser part de `self.joueur.bout_du_canon()` et va jusqu'à `self.cible.position`.

✓ Testez : lancez le jeu, vous devez voir...

Le jeu est jouable ! Tapez un mot : il recule à chaque lettre et disparaît quand il est complet. Le score reste à 0 pour l'instant (il sera calculé à l'étape 9). Les lasers et le cadre de la cible ne sont pas encore visibles : ce sont les étapes 6 à 8.

Correction

```
def gerer_frappe(self, lettre):
    self.frappes_totales += 1
    if self.cible is None:
        self.cible = self.chercher_cible(lettre)
        if self.cible is None:
            self.rater()
            return

    if self.cible.taper(lettre):
        self.frappes_correctes += 1
        self.joueur.viser(self.cible.position)
        self.lasers.append(Laser(self.joueur.bout_du_canon(),
                                 self.cible.position))
        if self.cible.est_termine():
            self.detruire(self.cible)
    else:
        self.rater()
```

Exercice 1 – Questions sur le ciblage 5 min

1. Dans `mettre_a_jour`, pourquoi faut-il écrire `if mot is self.cible: self.cible = None` quand un mot touche le vaisseau ?
2. Pourquoi `changer_de_cible` remet-il `tape` à 0 sur l'ancien mot ?
3. Quelle différence entre `mot is self.cible` et `mot == self.cible` ?

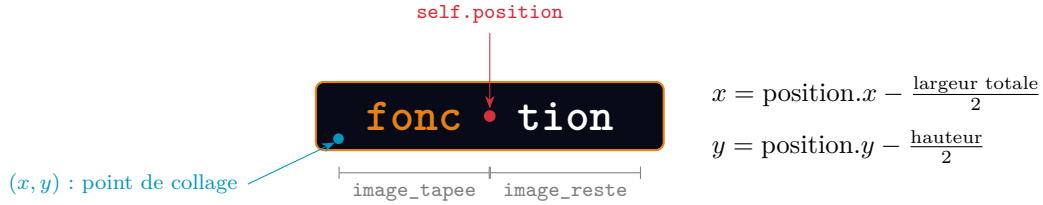
Correction

1. Sinon `self.cible` désignerait un mot qui n'est plus dans la liste : les frappes suivantes seraient « absorbées » par un mot fantôme invisible.
2. Pour pouvoir le re-cibler plus tard en tapant de nouveau sa *première* lettre.

3. `is` teste si c'est *le même objet* en mémoire ; `==` teste l'égalité des valeurs. Deux mots différents pourraient avoir le même texte : on veut *cet* objet-là.

Étape 6 – Afficher la progression TODO 6 — 10 min

`Mot.dessiner` doit montrer ce qui a déjà été tapé. On « rend » **deux images** : `texte[:tape]` en `ORANGE` et `texte[tape:]` en `couleur_reste` (blanc, ou rouge si le mot est proche). On les colle côte à côte, l'ensemble centré sur `self.position` :



Avant le texte, dessinez un rectangle de fond `CADRE` (6 px de marge, `border_radius=6`) ; si `est_cible`, entourez-le d'un cadre `ORANGE` d'épaisseur 2. Remarquez dans `dessiner_partie` que la cible est dessinée **en dernier** : pourquoi ?

✓ Testez : lancez le jeu, vous devez voir…

Les mots ont un fond sombre, la partie tapée est orange, la cible a un cadre orange, et les mots proches du vaisseau deviennent rouges.

Correction

```
image_tapee = police.render(self.texte[:self.tape], True, ORANGE)
image_reste = police.render(self.texte[self.tape:], True, couleur_reste)
largeur = image_tapee.get_width() + image_reste.get_width()
hauteur = police.get_height()
x = self.position.x - largeur / 2
y = self.position.y - hauteur / 2

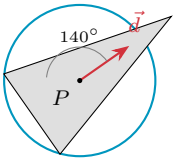
fond = pygame.Rect(x - 6, y - 2, largeur + 12, hauteur + 4)
pygame.draw.rect(ecran, CADRE, fond, border_radius=6)
if est_cible:
    pygame.draw.rect(ecran, ORANGE, fond, 2, border_radius=6)

ecran.blit(image_tapee, (x, y))
ecran.blit(image_reste, (x + image_tapee.get_width(), y))
```

La cible est dessinée en dernier pour passer au-dessus des mots qui la chevauchent.

Étape 7 – Orienter le canon TODO 7a TODO 7b — 10 min

1. TODO 7a : `Joueur.viser(point)` met dans `self.direction` le vecteur unitaire du joueur vers le point. **Attention** : normaliser un vecteur nul provoque une erreur, testez sa longueur d'abord.
2. TODO 7b : dessinez le triangle à partir de $\vec{d}$ = `self.direction` :
   - pointe $= P + 22\,\vec{d}$;
   - ailes $= P + 15\,\vec{d}$ tourné de $+140°$ et de $-140°$.



✓ Testez : lancez le jeu, vous devez voir...

Le vaisseau pivote vers chaque mot que vous tapez.

Correction

```
def viser(self, point):
    vecteur = point - self.position
    if vecteur.length() > 0:
        self.direction = vecteur.normalize()

def dessiner(self, ecran):
    pygame.draw.circle(ecran, CYAN, self.position, self.rayon, 2)
    d = self.direction
    pointe = self.position + d * 22
    gauche = self.position + d.rotate(140) * 15
    droite = self.position + d.rotate(-140) * 15
    pygame.draw.polygon(ecran, BLANC, [pointe, gauche, droite])
```

Étape 8 – Des lasers qui s'effacent TODO 8 — 10 min

Un `Laser` est un objet **à durée de vie limitée** : il naît avec `temps_restant = DUREE_LASER`, perd `dt` à chaque image, et `Jeu` le retire de la liste quand `est_fini()`. Ce schéma servira pour toutes les particules du TP1.
Pour qu'il s'efface, on **mélange** sa couleur avec celle du fond selon le `ratio` r = `temps_restant`/`DUREE_LASER`, qui passe de 1 à 0 (*interpolation linéaire*, composante par composante) :

$$\text{couleur} = \text{fond} + (\text{cyan} - \text{fond}) \times r \qquad \begin{cases} r = 1 : \text{cyan} \\ r = 0 : \text{couleur du fond (invisible)} \end{cases}$$

En Python, `zip(CYAN, FOND)` parcourt les deux tuples en parallèle. L'épaisseur vaut `1 + int(3 * ratio)`.

✓ Testez : lancez le jeu, vous devez voir...

Chaque bonne lettre tire un trait cyan du canon vers le mot, qui s'affine et s'estompe en un éclair.

Correction

```
def dessiner(self, ecran):
    ratio = max(0.0, self.temps_restant / DUREE_LASER)
    couleur = tuple(int(f + (c - f) * ratio) for c, f in zip(CYAN, FOND))
    epaisseur = 1 + int(3 * ratio)
    pygame.draw.line(ecran, couleur, self.depart, self.arrivee, epaisseur)
```

Étape 9 – Score et combo TODO 9a TODO 9b — 10 min

- Le **combo** compte les mots détruits d'affilée sans faute ni choc.
- Le **multiplicateur** gagne +1 tous les `COMBO_PAR_PALIER` (10) mots de combo, plafonné à `MULTIPLICATEUR_MAX` (4) : $m = \min(1 + \lfloor \text{combo}/10 \rfloor,\ 4)$. En Python, $\lfloor a/b \rfloor$ s'écrit `a // b` (division entière).
- Un mot rapporte $10 \times$ nombre de lettres $\times\ m$ points.
- Le niveau vaut $1 + \lfloor \text{mots_detruits}/10 \rfloor$.

`TODO 9a` : `multiplicateur()`. `TODO 9b` : dans `detruire(mot)`, mettez à jour `mots_detruits`, `combo`, `meilleur_combo`, `score`, puis le niveau ; s'il augmente, lancez le message « NIVEAU n » en mettant `self.message_niveau = DUREE_MESSAGE_NIVEAU`.

✓ Testez : lancez le jeu, vous devez voir…

Le HUD affiche score, niveau et combo. Au 10e mot sans faute, le combo passe en jaune « x2 » ; au 10e mot détruit, « NIVEAU 2 » s'affiche 1,5 s.

Correction

```python
def multiplicateur(self):
    return min(1 + self.combo // COMBO_PAR_PALIER, MULTIPLICATEUR_MAX)

# dans detruire(self, mot), après self.cible = None :
self.mots_detruits += 1
self.combo += 1
self.meilleur_combo = max(self.meilleur_combo, self.combo)
self.score += POINTS_PAR_LETTRE * len(mot.texte) * self.multiplicateur()

nouveau_niveau = 1 + self.mots_detruits // MOTS_PAR_NIVEAU
if nouveau_niveau > self.niveau:
    self.niveau = nouveau_niveau
    self.message_niveau = DUREE_MESSAGE_NIVEAU
```

Étape 10 – Précision et vitesse de frappe `TODO 10a` `TODO 10b` 5 min

— **Précision** $= 100 \times \dfrac{\text{frappes correctes}}{\text{frappes totales}}$ (100 si aucune frappe : attention à la division par zéro !).

— **Mots par minute** : par convention internationale, « un mot » vaut **5 frappes**, quelle que soit la longueur réelle des mots : MPM $= \dfrac{\text{frappes correctes}/5}{\text{temps en minutes}}$. Renvoyez 0 pendant la première seconde (sinon on diviserait par un temps quasi nul).

✓ Testez : lancez le jeu, vous devez voir…

En bas de l'écran, la précision baisse quand vous faites des fautes ; la vitesse se stabilise après quelques secondes (un débutant tape 20 à 30 MPM, un bon dactylo plus de 60).

Correction

```python
def precision(self):
    if self.frappes_totales == 0:
        return 100.0
    return 100 * self.frappes_correctes / self.frappes_totales

def mpm(self):
    if self.temps_jeu < 1:
        return 0.0
    return (self.frappes_correctes / 5) / (self.temps_jeu / 60)
```

Étape 11 – Une difficulté progressive `TODO 11a` `TODO 11b` `TODO 11c` 15 min

Trois leviers rendent le jeu plus dur avec le niveau n :

— **la longueur des mots** : `TODO 11a` `mots_du_niveau(n)` renvoie la réserve de mots : niveaux 1–2 courts ; 3–4 courts + moyens ; 5–6 moyens + longs ; 7 et plus longs + très longs (on concatène deux listes avec `+`).

— **la vitesse**, qui croît *linéairement* (suite arithmétique) : `TODO 11b` $v_n = v_1\,(1 + 0{,}12\,(n-1))$;

— **le délai entre deux mots**, qui décroît *géométriquement*, avec un plancher : `TODO 11c` $d_n = \max\,(0{,}55;\ 2{,}4 \times 0{,}88^{\,n-1})$.

Niveau n	1	2	3	4	5	6	7	8	10	13
Vitesse (px/s)	45	50	56	61	67	72	77	83	94	110
Délai (s)	2,40	2,11	1,86	1,64	1,44	1,27	1,11	0,98	0,76	0,55
Mots / minute à l'écran	25	28	32	37	42	47	54	61	79	109

Question : pourquoi un délai géométrique plutôt que linéaire ($d_n = 2{,}4 - 0{,}2\,(n-1)$) ?

✓ Testez : lancez le jeu, vous devez voir…

Pour tester vite, passez provisoirement `MOTS_PAR_NIVEAU` à 2 : les mots s'allongent, accélèrent et arrivent de plus en plus souvent. Remettez 10 ensuite.

Correction

```python
def mots_du_niveau(niveau):
    if niveau <= 2:
        return MOTS["courts"]
    if niveau <= 4:
        return MOTS["courts"] + MOTS["moyens"]
    if niveau <= 6:
        return MOTS["moyens"] + MOTS["longs"]
    return MOTS["longs"] + MOTS["tres_longs"]

def vitesse_actuelle(self):
    return VITESSE_BASE * (1 + GAIN_VITESSE * (self.niveau - 1))

def delai_actuel(self):
    return max(DELAI_MIN, DELAI_BASE * FACTEUR_DELAI ** (self.niveau - 1))
```

Un délai linéaire deviendrait *négatif* au niveau 13 et la difficulté exploserait d'un coup ; le délai géométrique diminue de 12 % à chaque niveau : la progression *ressentie* est régulière.

Étape 12 – Des initiales uniques `TODO 12` 10 min

Si deux mots à l'écran commencent par la même lettre, taper cette lettre est ambigu. Lors de l'apparition, on préfère donc un mot dont l'initiale n'est pas déjà utilisée :

1. construire l'**ensemble** (`set`) des initiales présentes : `{mot.texte[0] for mot in self.mots}` ;
2. garder les mots `libres` de la réserve dont l'initiale n'est pas dans cet ensemble (liste en compréhension) ;
3. tirer dans `libres` s'il n'est pas vide, sinon dans toute la réserve.

Pourquoi un `set` plutôt qu'une liste pour les initiales ?

Correction

```
initiales = {mot.texte[0] for mot in self.mots}
libres = [m for m in reserve if m[0] not in initiales]
texte = random.choice(libres if libres else reserve)
```

Le test `in` sur un `set` est quasi instantané quelle que soit sa taille, alors que sur une liste Python doit la parcourir. Et un `set` élimine les doublons tout seul.

Le jeu est complet ! Le fichier `td2_correction.py` sera distribué comme base commune des TP.

4 Pour aller plus loin (si vous avez fini)

1. **Faute visible :** une faute fait clignoter la cible en rouge pendant 0,15 s.
2. **Bonus de vitesse :** un mot détruit en moins de 1 s après son verrouillage rapporte le double.
3. **Pause :** ajoutez un état `"pause"` (touche `Tab`) au diagramme et au code.
4. **Barre de progression :** une barre en bas de l'écran montre l'avancée vers le niveau suivant.

5 Backlog : tout ce qu'on peut ajouter au jeu

Dans un studio, la liste des fonctionnalités à faire s'appelle le *backlog*. Chaque tâche est petite, testable, et a une difficulté estimée (⋆ facile, ⋆⋆ moyen, ⋆⋆⋆ difficile). Les TP piochent dans cette liste ; cochez ce que vous avez réalisé. Les tâches marquées TP1, TP2, TP3 sont celles proposées en priorité à chaque séance.

Effets visuels (*game feel*)

☐ **Explosion de particules** ⋆⋆ TP1
À la destruction d'un mot, 20 à 40 petits points partent dans toutes les directions, ralentissent et s'effacent. Une classe `Particule` avec une durée de vie, comme `Laser`.

☐ **Tremblement d'écran** ⋆ TP1
Quand le bouclier est touché, tout le dessin est décalé d'un vecteur aléatoire qui diminue en 0,3 s.

☐ **Ciel en parallaxe** ⋆⋆ TP1
Trois couches d'étoiles qui défilent à des vitesses différentes donnent une impression de profondeur.

☐ **Nébuleuse et planètes** ⋆⋆
Un fond pré-dessiné une fois sur une `Surface` : dégradé, grands disques semi-transparents, planète avec anneau.

☐ **Canon qui tourne en douceur** ⋆⋆ TP1
Au lieu de pointer instantanément, le canon tourne vers sa cible à vitesse angulaire limitée (`Vector2.angle_to`).

☐ **Lettres qui volent** ⋆⋆
Chaque lettre tapée se détache du mot et tombe en tournant.

☐ **Bouclier qui clignote** ⋆
Une seconde d'invulnérabilité après un choc, avec clignotement.

☐ **Couleur du combo** ⋆
Le vaisseau et les lasers changent de couleur selon le multiplicateur.

☐ **Images (sprites)** ⋆
Remplacer le triangle par une image `.png` (`pygame.image.load`, `pygame.transform.rotate`).

Son

☐ **Bruitages** ⋆ TP1
Un son pour chaque frappe juste, faute, destruction, choc, niveau (`pygame.mixer.Sound`).

☐ **Musique de fond** ⋆
`pygame.mixer.music` en boucle, coupée dans le menu pause.

☐ **Sons synthétisés** ⋆⋆⋆
Générer les sons par calcul (sinusoïde, bruit) sans aucun fichier.

Mots et langue

☐ **Dictionnaire externe** ⋆ TP1
Charger les mots depuis `mots.txt` (`open`, `split`), avec la liste du code en secours si le fichier manque.

☐ **Accents** ⋆⋆ TP1
Accepter « é » et « e » (normalisation avec `unicodedata`), ou exiger l'accent en mode difficile.

☐ **Thèmes de mots** ⋆
Espace, Python, informatique, capitales… choisis dans le menu.

☐ **Majuscules** ⋆⋆
En mode expert, la casse compte (`Maj` obligatoire).

Ennemis

☐ **Mot rapide** ⋆ TP2
Court, deux fois plus rapide, affiché en jaune.

☐ **Mot blindé** ⋆⋆ TP2
Il faut le taper deux fois (un 2e mot apparaît quand le 1er est détruit).

☐ **Mot diviseur** ⋆⋆ TP2
En mourant, il libère deux petits mots à sa place (héritage : `class MotDiviseur(Mot)`).

☐ **Mot zigzag** ⋆⋆
Trajectoire sinusoïdale autour de sa direction (`math.sin(temps)`).

☐ **Mot fantôme** ⋆⋆
Il devient invisible quelques secondes : il faut s'en souvenir.

☐ **Boss-phrase** ⋆⋆⋆ TP2
Tous les 5 niveaux, un vaisseau géant affiche une *phrase* (avec espaces) et envoie des petits mots. Une barre de vie montre la progression.

Bonus

☐ **Mot bonus « soin »** ⋆ TP2
Vert ; le détruire rend une vie.

☐ **Mot bonus « bombe »** ⋆ TP2
Le détruire détruit tous les mots à l'écran.

☐ **Mot bonus « gel »** ⋆⋆
Ralentit tous les mots pendant 5 s (un facteur de temps appliqué à `dt`).

☐ **Mot bonus « bouclier »** ⋆⋆
Absorbe le prochain choc.

Progression

☐ **Vagues** ⋆⋆ TP2
Un nombre fixe de mots par vague, une pause entre deux vagues.

☐ **Améliorations entre les vagues** ⋆⋆⋆ TP2
Comme dans Glyphica : choisir 1 amélioration parmi 3 tirées au hasard (vie max, recul, multiplicateur, ralentissement…).

☐ **Modes de jeu** ⋆⋆
Survie (infini), Contre-la-montre (60 s), Zen (pas de vies).

☐ **Niveaux de difficulté** ⋆ TP3
Facile / Normal / Difficile modifient les constantes de difficulté.

Interface et données

☐ **Pause** ⋆ TP3
Un nouvel état `"pause"` dans la machine à états (touche `Tab`).

☐ **Meilleurs scores sauvegardés** ⋆⋆ TP3
Top 10 dans un fichier `scores.json` (`json.load`/`json.dump`), avec `try/except` si le fichier n'existe pas.

☐ **Saisie du pseudo** ⋆⋆ TP3
Un état `"saisie_nom"` après un record.

☐ **Statistiques détaillées** ⋆⋆ TP3
En fin de partie : lettres les plus ratées (dictionnaire de compteurs), meilleur mot par minute, histogramme dessiné avec `pygame.draw.rect`.

☐ **Menu d'options** ⋆⋆
Volume, plein écran, thème, navigation aux flèches.

☐ **Tutoriel** ⋆
Un premier niveau commenté pour les nouveaux joueurs.

Qualité du code

☐ **Docstrings et constantes** ⋆ TP3
Chaque classe et méthode documentée, aucun « nombre magique » hors des constantes.

☐ **Tests de la logique** ⋆⋆
Tester `Mot.taper`, `multiplicateur`, `precision` avec `assert`, sans ouvrir de fenêtre.

☐ **Mode debug** ⋆
`F1` affiche FPS, nombre de mots, cercles de collision.

6 Feuille de route des TP (10 h)

Chaque binôme part de `td2_correction.py` et le fait évoluer, **toujours dans un seul fichier**. À chaque séance, on choisit des tâches dans le backlog. Un minimum est attendu, le reste est libre.

Séance	Minimum attendu	Pour aller plus loin
TP1 (4 h) *Game feel*	explosions de particules ; tremblement d'écran ; au moins 3 bruitages ; mots chargés depuis `mots.txt`	parallaxe, nébuleuse, canon qui tourne en douceur, accents
TP2 (3 h) *Gameplay*	2 types d'ennemis spéciaux (classes héritant de `Mot`) ; 2 bonus	boss-phrase tous les 5 niveaux, vagues, écran d'améliorations
TP3 (3 h) *Finition*	pause ; top 10 sauvegardé en JSON ; code documenté	pseudo, statistiques détaillées, options, modes de jeu

Rendu : le fichier `.py` unique (et ses éventuels fichiers `mots.txt`, sons, images), avec en en-tête la liste des fonctionnalités ajoutées. **Évaluation indicative :** fonctionnement et absence de plantage (30 %), fonctionnalités réalisées (35 %), qualité du code — classes, constantes, docstrings (25 %), originalité (10 %).

Bilan de la séance

Ce qu'il faut retenir

- Une **machine à états** (`self.etat` + transitions) structure menus, partie et fin.
- Pour taper du texte : `KEYDOWN` + `evenement.unicode`, filtré par `isalpha()`.
- Chaînes : `texte[i]`, `texte[:i]`, `texte[i:]`, `len(texte)`.
- Recherche du minimum : une variable « meilleur » initialisée à `None` / `math.inf`.
- Objets à durée de vie : un compteur qui décroît de `dt`, puis on filtre la liste.
- Interpolation linéaire : $a + (b - a) \times r$ pour mélanger deux valeurs.
- Difficulté : progression **linéaire** pour la vitesse, **géométrique** avec plancher pour le délai.
- `is` compare des objets, `==` compare des valeurs ; un `set` teste l'appartenance très vite.